

Kod, który słyhać – program warsztatu

Sześć kroków w trzy godziny; każdy odpowiada na pytanie, które zostawił poprzedni. Każdy krok ma gałąź `step-N` (szkielet, czerwone testy) i `step-N-final` (rozwiązanie); klasa startowa leży w pakiecie `pl.livecoding.musicjam.stepN`.

krok	co się dzieje	uczestnicy piszą	min	start
0	otwarcie: plan, <code>javax.sound.midi</code> / <code>sampler</code> , Java kontra MIDI	–	5	0:00
1	<code>SequencerDemo</code> : Java gra plik MIDI za nas	nic	5	0:05
2	<code>InspectMidi</code> : rozbieramy plik na zdarzenia	parowanie nut, tick → beat	25	0:10
3	<code>PlayNotes</code> : własny odtwarzacz przez Gervill	wyjscie MIDI, lista zdarzeń, scheduler z pulą	30	0:35
–	przerwa		10	1:05
4	ta sama melodia na zewnętrznym synteźniku	wyjscie MIDI na prawdziwe urządzenie	20	1:15
5	perkusja co do próbki, dwa zegary, jeden zegar	harmonogram w ramkach, poprawka zegara	45	1:35
6	live coding w studiu	nic	15	2:20
–	powrót do kroku 1, podsumowanie		5	2:35

Razem 160 min, 20 min zapasu. Kto utknie: `git switch step-N-final` albo ściąga z rozwiązań – i dalej razem z grupą.

1. [Przed startem](#)
2. [Krok 0 · Otwarcie](#)
3. [Krok 1 · SequencerDemo](#)
4. [Krok 2 · InspectMidi](#)
5. [Krok 3 · PlayNotes](#)
6. [Krok 4 · PlayOnSynth](#)
7. [Krok 5 · PlayBeat, PlayJam](#)

8. [Krok 6 · Studio](#)

9. [Podsumowanie](#)

Przed startem

- JDK 25 (Gradle sam pobierze toolchain), słuchawki, repo sklonowane i zbudowane.
- Uruchamiamy klasy z IntelliJ. Lepiej przez IntelliJ niż przez Gradle: Settings → Build, Execution, Deployment → Build Tools → Gradle → „**Build and run using: IntelliJ IDEA**”. Wtedy Stop wykonuje hak zamknięcia i zewnętrzny syntezytor milknie.
- Na krok 4: Surge XT i wirtualny kabel – loopMIDI (Windows: uruchomić, dodać port) albo IAC Driver (macOS: Audio MIDI Setup → MIDI Studio → IAC Driver → „Device is online”). W Surge kabel jako wejście MIDI.
- Syntezytor sprzętowy (np. KORG NTS-1 po USB): `midiDevice=NTS-1`, a kanał sprawdzić – NTS-1 ustawiony na „1” grał na `midiChannel=2`.
- U siebie zmierzyć `midiLatency` kalibratorem: Surge XT przez loopMIDI 35–50 ms, Mac ok. 75 ms, Gervill ponad 200 ms. Domyślnie 50.
- Slajdy, ściąga z rozwiązań i ten program na tablecie.

0 · Otwarcie

· 5 min

- Pomysł: kod, który słyhać – czas muzyczny w Javie, od gotowego odtwarzacza do własnego silnika.
- W JDK są dwa pakiety i wyznaczają program: `javax.sound.midi` (kroki 1–4) – co i kiedy zagrać, zdarzenia, nie dźwięk; `javax.sound.sampled` (kroki 5–6) – sam dźwięk, próbki na kartę. Bez bibliotek.
- Tabela Java kontra MIDI: następca (MIDI 2.0 w 2020, Kotlin i Scala – oryginały mają się dobrze), kompatybilność wsteczna (klawiatura z 1983 steruje Surge XT, plik `.class` z Javy 1.0 działa na JVM 25), co się zmieniło (kabel i platforma, nie protokół i język), ogłaszane martwe co roku.

1 · Czarna skrzynka – SequencerDemo

· 5 min

Pokaż: `step1.SequencerDemo` – cztery linie, `Sequencer` gra cały plik z `jam.properties`.

- Moment, w którym wszyscy sprawdzają dźwięk, słuchawki i build.
- `Config` zamienia argumenty i plik `.properties` na ścieżkę; każdy kolejny krok dokłada pole.

Na dalej: co właściwie robi `Sequencer` – i czy umiemy lepiej?

Pokaż: `step2.InspectMidi` z `--config src/main/resources/jam-dre.properties`: lista ścieżek (nazwa, kanał, program GM, liczba nut, zakres, pierwszy takt), potem nuty w beatach. `--overview` – sama lista.

Powiedz

- **Tick** → **beat**. PPQ = ticki na ćwierćnutę (`getResolution()`). `song_shape.mid`: PPQ 384, tick 3072 = beat 8.0 = takt 3. Pomocnik `ticksToBeats` jest gotowy.
- **Note-on** → **note-off**. Nuta to para zdarzeń; note-on z velocity 0 to w praktyce note-off. `PlayingNotes` trzyma kolejkę na każdą wysokość – ta sama nuta może zabrzmieć znowu przed zwolnieniem poprzedniej.
- **Plik jest zagrany, nie skwantyzowany**: w Dre riff to trzy nuty naraz, a górna jest stale 0,063 beatu później. Niczego nie zaokrąglamy.
- **Tempo w pliku to tylko deklaracja**: *Insomnia* miała nuty w siatce 130 BPM, a plik mówił 125 – riff trwał 7,69 beatu zamiast 8. `song_insomnia.mid` ma poprawione tempo i rozciągnięte ticki; każda nuta brzmi w tej samej sekundzie co przedtem.

Piszą

Gałąź komunikatów kanałowych w `MidiFileReader.readTrack` (kanał, program, note-on/off), sortowanie nut, `noteBetween`. Gotowe: PPQ, jedno przejście po ścieżce, meta (nazwa, tempo). `MidiFileReaderTest`: 9 testów, 4 zielone od startu, bez pliku i bez dźwięku.

Na dalej: mamy `List<Note>` w beatach – a `Sequencer` umie zagrać tylko `Sequence`.

Pokaż: `step3.ListMidiDevices` – pierwszy na liście jest Gervill, syntezytor z JDK, ten sam, którego używał `Sequencer`. Zmienia się tylko to, kto planuje nuty.

Piszą, w kolejności

- `MidiNoteOutput`: `open` (otwórz syntezytor, kanał, `programChange`), `noteOn`, `noteOff`. Gotowe: `channelOf`, rozgrzewka, zamykanie. Test na atropie, bez dźwięku.
- `MidiPlayer.schedule`: każda nuta to dwa zdarzenia, każde z **momentem liczonym od startu**, nie z opóźnieniem od poprzedniego – spóźnienia się nie sumują.
- `PooledScheduler.submit`: cztery linie; ciekawa jest zamiana momentu na opóźnienie „od teraz”, którego chce `ScheduledExecutorService`.

Pokaż: `--scheduler all` – platform, virtual, pool, scoped; każdy kończy się `TimingReport` (przygotowanie, spóźnienie wybudzeń).

Powiedz

- Żaden nie trafia w moment: `Thread.sleep` wraca, kiedy system operacyjny zdąży. Wątki wirtualne tanio czekają, ale nie są dokładniejsze.
- `TimingReport` mierzy wybudzenie, nie dźwięk – Gervill ma jeszcze własny bufor.
- `scoped` warto przeczytać: żadne zadanie nie przeżyje zakresu. `StructuredTaskScope` to wciąż preview w Javie 25 – stąd `--enable-preview` i przypięty JDK.
- Pierwsza nuta w Gervillu potrafi się zaciąć – instrument ładuje się leniwie; dlatego `warmUp`.

Przerwa, 10 min: dobry moment, żeby każdy uruchomił Surge XT i kabel MIDI.

Pokaż: `ListMidiDevices` – `loopMIDI` (i `NTS-1`) widać dwa razy pod jedną nazwą: końcówka, do której wysyłamy, i ta, z której czyta syntezytor; wiadomości przyjmuje tylko pierwsza. W configu `midiDevice=loopMIDI` (dowolny fragment nazwy). Potem `step4.PlayOnSynth` – wypisuje urządzenie i kanał.

Piszą

`ExternalMidiOutput` – drugi `NoteOutput`: `find` (pierwsze pasujące urządzenie, które przyjmuje wiadomości), `noteOn/noteOff/send` (`ShortMessage` do `Receiver`, znacznik czasu `-1` = „teraz”), `allSoundOff`. `ExternalMidiOutputTest`: 9 testów, 3 zielone od startu (otwieranie, panic przy otwarciu, blokada note-onów).

Powiedz

- **Kanał.** Nuty idą na kanale ścieżki; syntezytor sprzętowy słucha jednego kanału. `midiChannel=` (1–16, jak liczy syntezytor) go nadpisuje; `javax.sound.midi` liczy od 0. `NTS-1` z „1” na wyświetlaczu grał na `midiChannel=2`.
- **CC 120 kontra CC 123.** `All Sound Off` ucina dźwięk od razu; `All Notes Off` tylko zwalnia nuty, pad z długim ogonem gra dalej.
- **Nic nie zostaje wiszące.** Przy zamknięciu i w haku (`Ctrl+C`) najpierw blokada note-onów – wątki schedulera działają jeszcze w trakcie zamykania JVM – potem `All Sound Off` i note-off dla każdej wysokości. Przy otwarciu urządzenie jest najpierw wyciszane.
- **Stop w IntelliJ.** Przy uruchomieniu przez Gradle zabija JVM bez haków – nuta gra dalej. Pomaga „Build and run using: IntelliJ IDEA”, kolejne uruchomienie albo `step4.MidiPanic`.
- `--scheduler all` daje te same liczby co z Gervillem, a Surge i tak gra nutę dopiero po swoim buforze. Tego nie zmierzmy – dźwięk powstaje w innym programie, na innym zegarze.

Komu nie działa kabel lub syntezytor: `--midiDevice Gervill`, ten sam kod; w kroku 5 niczego nie traci.

Na start (gotowe, bez ćwiczeń)

- `step5.CalibrateLatency` – stopa z `AudioEngine` i nuta z syntezy na każdy beat; suwak (0–400 ms) aż zleją się w jedno uderzenie; linię `midLatency=` do configu.
- `step5.ShowSamples` – każdy bęben dwa razy: z pliku i ze wzoru. Pliki są różne (`bd.wav` 192 kHz stereo, `sd.wav` 44,1 kHz stereo, `hh.wav` mono 24 bit, `oh.wav` i `cp.wav` stereo 24 bit); `WavSampleLoader` robi z nich 44,1 kHz mono. Dla renderera to tylko tablica liczb.

Powiedz

Przestajemy czekać na moment – liczymy ramkę. Karta dostaje bloki po 512 ramek (ok. 12 ms). W 120 BPM szesnastka to 5512,5 ramki – każdą pozycję liczymy od początku, więc zaokrąglenia się nie sumują.

Piszą, w kolejności testów

- `Transport.frameAtBeat` (`TransportTest`).
- `PatternCompiler.compile`: takt "X...x...o..." na te same `Note`, co z pliku; `parse` gotowe.
- `AudioEngine.schedule` – serce kroku: harmonogram z kroku 3 w ramkach. `Hit` niesie całą nutę: `hits.add(new Hit(frame, note))`. `AudioEngineTest` (1000 ramek/s, bloki 128) zazielenia się razem z nim.

Pokaż: w `renderNext` zamień `hits[nextHit].frame()` na `position` i puść `PlayBeat --block 4096` (93 ms) – groove się rozsypuje. Przywróć linię.

Dwa zegary

`step5.PlayJam` z `jam-insomnia.properties` (8 taktów, perkusja `insomnia`, ściszone o połowę) – perkusja idzie za kartą, melodia za `System.nanoTime()`. Z `--stall 300` perkusja po 3 s staje jak przy pauzie GC – melodia już jej nie dogoni.

Piszą: pętlę w `playLoop`: przed każdą pętlą zapytać gotowe `loopStartNanos` (teraz, `heardNanos`, numer pętli, długość, opóźnienie) i oddać ją `playOneLoop`. `MidiPlayerLoopTest`: arytmetyka zielona od startu, potem trzy pętle przeciw „audio”, które spóźnia się o 100 ms.

Jeden zegar (gotowe, 5 min pokazu)

```
PlayJam --config src/main/resources/jam-insomnia.properties --synth pluck
```

- `synth=anthem|pluck|pad`: melodia nie idzie przez MIDI – `AudioEngine` renderuje każdą nutę przez syntezytor na długość nuty i kładzie ją na jej ramce jak próbkę bębna.

- Nie ma `midilatency` ani drugiego zegara; z `--sta11 300` perkusja i melodia pauzują razem.

Puenta: dwóch zegarów nie ustawia się raz – porównuje się je w kółko. Albo zostaje jeden zegar – krok 6.

6 · Zmiany w trakcie grania – studio

· 15 min

Pokaż: gałąź `final`, `StudioLauncher` (`./gradlew studio`). Jeden `LiveSession` renderuje perkusję i melodię na jednym zegarze.

- Presety jamów, w tym `Insomnia`; siatka kroków, tempo, długość pętli aż do 1/16 taktu, melodia on/off.
- Zewnętrzny syntezytor: kanał z `midiChannel=`, filtr (CC 74), suwak opóźnienia do 400 ms – słycać od razu.
- Edytor w stylu Strudla (podzbiór):

```
$: stack(  
    s("bd(3,8,5)"),  
    s("[~ sd]*2").gain(1.25),  
    s("hh*16").gain("[0.2 0.1]*8")  
)
```

Ctrl+Enter wgrywa; `//` wycisza warstwę; kod z błędem się nie wgrywa – gra poprzedni wzór.

- Każda zmiana wchodzi na granicy pętli – w pętli każde uderzenie nadal trafia w swoją ramkę.
- `LiveCode.notes` to trzeci sposób na `Note`: po pliku MIDI i wzorze – tekst pisany w trakcie grania.

Podsumowanie – powrót do kroku 1

· 5 min

- Teraz wiemy, co robi `Sequencer`: czyta plik (krok 2), planuje zdarzenia na swoim wątku (3) i oddaje je syntezytorowi – Gervillowi albo zewnętrznemu (4).
- Czekanie na moment zawsze się spóźnia; liczenie ramki trafia co do próbki (5).
- Dwa zegary trzeba porównywać; jeden zegar załatwia sprawę (5–6).
- Wszystko na samym JDK: `javax.sound.midi` + `javax.sound.sampled`.